

다음 작업 — 시뮬레이션이 알아낸 것을 실시간 판단에 먹이기

zone_base_risk 연결 · 2026-08-06 작성 · 착수 시점 8/9 이후

1. 지금 무엇이 문제인가

젯슨에서 두 흐름이 돌고 있는데 서로 연결되어 있지 않다. 시뮬레이션이 3초 앞을 예측해도 그 결과가 지팡이 판단에 아무 영향을 주지 않는다. 지도에만 그려지고 끝난다.

[시뮬레이션 흐름] GCP 서버 SUMO 생성 → 젯슨 process_scenarios.py (cron 1분)
→ v3 트랜스포머 3초 예측 → 서버 업로드 → 위험지도

[실시간 흐름] 노드 GPS 5Hz → 브리지 → 젯슨 step8_send_risk.py (systemd)
→ 규칙 + 안전하한 + GBM → 지팡이 진동 (0.1초)

간접 경로는 하나 있다. 시뮬레이션 데이터로 모델을 학습해 risk_model.json으로 젯슨에 넣었으므로, 시뮬레이션이 "판단하는 법"은 이미 전달되고 있다. 없는 것은 운영 중에 계속 갱신되는 지식이다.

2. 받을 자리는 이미 있다

step7_risk.py의 팀 점수표에 구간 위험도를 받는 인자가 설계되어 있다.

```
def calculate_risk_score(distance_m, relative_speed_mps,
    vehicle_speed_mps, ttc, zone_base_risk=0):
    ...
    # 5. Hazard-zone correction: up to 5
    score += min(max(zone_base_risk, 0), 5)
```

그런데 step8이 이 값을 넘기지 않아 항상 0이다. 100점 중 5점을 쓸 수 있는데 비워두고 있는 셈이다. auto_pipeline/process_scenarios.py도 "SUMO에 구역 정보가 없으므로 0을 쓴다"고 주석에 적어두었다.

3. 만들 구조

SUMO 시뮬 + 실측 이벤트
↓
위험지도 API 구간별 집계 ← 이미 가동 중
↓ 젯슨이 주기적으로 내려받아 캐시
현재 GPS → 구간 매칭
↓
zone_base_risk → step7 점수표 → 위험 구간에서 더 일찍 경보

이렇게 되면 "데이터가 쌓일수록 지팡이가 똑똑해진다"가 성립한다. 서버가 넓게 보고 엣지가 빠르게 반응하는 협력 구조이고, 발표에서도 시스템이 하나로 도는 그림이 된다.

이미 있는 것: 젯슨 → 서버 이벤트 업로드(deploy/upload_events.py, 1분 타이머), 위험지도 API(Cloud Run), 구간 집계.
없는 것은 서버 → 젯슨 방향뿐이다.

4. 지금 바로 넣으면 안 되는 이유

근거 없는 가중치가 되기 때문이다. "교차로니까 +5점"을 주려면 교차로에서 실제로 더 자주 위험하다는 증거가 있어야 한다. 지금 가진 구간 위험도는 SUMO가 만든 값이고, 그 SUMO는 12,621스텝에서 위험 사례가 0건이었던 바로 그 시뮬레이터다.

이번 연구가 zone을 의도적으로 뺀 것도 같은 이유다. 오라클 라벨은 물리적 충돌만 보므로, 교차로에서 실제로 더 자주 충돌하도록 모델링하지 않는 한 zone 가중치는 정당화되지 않는다(SPEC.md 12절).

따라서 구조는 먼저 만들되 값은 데이터가 채우게 한다. 실측 이벤트가 쌓이면 "이 구간에서 위험이 N번 발생했다"는 통계적 근거가 생기고, 그때 값을 넣으면 된다.

5. 실행 단계

단계	할 일	판정 기준	담당
1	위험지도 API가 구간별 위험도를 내려주는 엔드포인트가 있는지 확인. 없으면 추가 요청 (GET /api/zones 형태)	구간ID → 위험도 JSON을 받을 수 있다	위험지도 담당
2	젯슨용 다운로더 작성. 주기적 갱신(10~30분), 로컬 캐시, 오프라인이면 마지막 캐시 사용	네트워크가 끊겨도 엔진이 멈추지 않는다	현준
3	현재 GPS → 구간 매칭. 구간 ID 형식은 1381099007_3 계열	실측 로그의 좌표가 올바른 구간으로 매칭된다	현준
4	step8 → step7에 zone_base_risk 전달. gate_params에 추가	기존 테스트 255개 통과 유지	현준
5	효과 검증 (아래 6절). 검증 전에는 값을 0으로 두고 로깅만	오라클 기준으로 성능이 오르는가	현준·민서

1~4단계는 배관 공사이고 위험이 없다. zone_base_risk를 0으로 두면 지금과 똑같이 동작하기 때문이다. 실제로 값을 넣는 것은 5단계 검증 이후다.

6. 어떻게 검증할 것인가

두 가지 방법이 있고, 둘 다 하는 것이 좋다.

6.1 시뮬레이션 검증 — 구조가 옳은지

scenario_sim.py에 구간 개념을 넣는다. 시나리오마다 "위험 구간" 표식을 부여하되, 표식이 있는 구간에서 실제로 위험이 더 자주 발생하도록 생성 파라미터를 다르게 준다(예: miss_offset 분포를 좁힌다). 그러면 zone 정보가 진짜 정보가 되고, 그것을 쓴 판정이 나아지는지 evaluate.py로 잴 수 있다.

이 실험은 "구간 정보가 있으면 도움이 되는가"라는 질문에 답한다. 도움이 되지 않는다면 배관만 만들어두고 값을 넣지 않으면 된다.

6.2 실측 검증 — 값이 옳은지

실측 이벤트가 쌓이면 구간별 발생 빈도를 집계한다. 특정 구간에 이벤트가 몰린다면 그 구간에 가중치를 주는 것이 정당해진다. 몰리지 않는다면 zone_base_risk는 0으로 두는 것이 맞다.

빈도 → 점수 환산은 단순하게 시작한다. 예를 들어 상위 10% 구간에 5점, 상위 30%에 3점, 나머지 0점. 복잡한 공식은 근거가 생긴 뒤에 만든다.

7. 착수에 필요한 정보

항목	내용
위험지도 API	https://riskmap-api-193571596396.asia-northeast3.run.app (Cloud Run, GCP 프로젝트 hanium-ssu-kpc). 팀원 관리 영역
기존 업로드 경로	deploy/upload_events.py — 레벨 1+ 이벤트를 POST /api/events, 1분 타이머. 인증키는 /etc/default/v2x-riskmap
집계 갱신	POST /api/admin/refresh-stats (빈 바디 -d "{}" 필요, 없으면 411). 지도는 집계 테이블 기준이라 이 호출 전에는 새 이벤트가 반영되지 않는다
구간 ID 형식	1381099007_3 계열
점수표 상한	zone_base_risk는 0~5로 클램프됨 (100점 중 5점)
관련 파일	step7_risk.py(계산), step8_send_risk.py(전달 지점), auto_pipeline/process_scenarios.py(시뮬레이션 쪽 동일 인자)

8. 이 작업을 8/9에 함께 논의해야 하는 이유

8/9에 "두 AI 모델을 어떻게 합칠 것인가"를 결정하기로 되어 있는데, 이 작업이 같은 질문의 다른 얼굴이다. 둘 다 "시뮬레이션 쪽과 실시간 쪽을 어떻게 연결할 것인가"를 묻고 있다.

질문	무엇을 연결하는가
두 모델 중 무엇을 쓸 것인가	시뮬레이션에서 학습한 것 → 실시간 판단
zone_base_risk를 어떻게 채울 것인가	시뮬레이션·실측에서 관측한 것 → 실시간 판단

따로 결정하면 서로 어긋난 구조가 나올 수 있다. 한자리에서 정하는 것이 좋다.

9. 예상 작업량

단계	작업량
1~4 (배관)	반나절. 위험이 없고 기존 동작 불변
6.1 시뮬레이션 검증	반나절. scenario_sim에 구간 개념 추가 + 재측정
6.2 실측 검증	데이터 축적 후. 집계 스크립트 자체는 1시간

정리하면 — 배관을 먼저 놓고(반나절), 구조가 옳은지 시뮬레이션으로 확인하고(반나절), 값은 실측이 쌓이는 대로 채운다. 급하지 않지만 연결하지 않으면 시뮬레이션 파이프라인이 지도 그리는 용도로 끝난다.